TRAITEMENT DE DONNÉES · FICHE DE RÉVISION · COURS 1

# Introduction au machine learning

Types d'apprentissage, train / validation / test, sur-apprentissage, représentation, optimisation, métriques

**À RETENIR EN PRIORITÉ**

- **Supervisé** (vérité terrain) : régression = sortie **quantitative**, classification = sortie **catégorielle**. **Non supervisé** : clustering, détection d'anomalie.
- **Paramètres μ** = appris automatiquement sur train. **Hyperparamètres λ** = fixés avant (degré, k, régularisation α, choix de la méthode) → choisis sur **validation**.
- **60 / 20 / 20** : train apprend (extraction **et** modèle), validation choisit les hyperparamètres, test = **une seule fois à la fin**. Rien n'est réappris sur validation / test.
- Erreur train **baisse toujours** avec la complexité ; erreur validation en **creux** : à gauche sous-apprentissage, à droite sur-apprentissage.
- Précision = TP / (TP + FP) · Rappel = TP / (TP + FN) · Spécificité = TN / (TN + FP) · $F_1 = 2PR / (P + R)$.
- Biais fort = modèle trop simple ; variance forte = modèle trop complexe. EQM = Biais² + Var.

## 1. Les types d'apprentissage

**Machine learning** : résoudre un problème automatiquement à partir d'**exemples**. La **prédiction** = réponse sur une **nouvelle donnée** (valeur, catégorie, groupe… pas forcément le futur).

| Type | Données d'apprentissage | Applications | Exemples du cours |
|---|---|---|---|
| **Supervisé** | Couples (donnée, prédiction attendue) = **vérité terrain**, données labellisées | **Régression** (sortie quantitative) **Classification** / discrimination (catégorielle) | Âge depuis une photo (régression) ; spam / non-spam, diagnostic, prix immobilier, visage |
| **Non supervisé** | Données seules, sans vérité terrain | **Clustering** (groupes homogènes) **Détection d'anomalie** | Regrouper des clients / acheteurs ; fraude bancaire, alarme de vidéosurveillance |
| **Semi-supervisé** | Petite partie labellisée **à la main**, grande partie non | Labelliser coûte cher, les données brutes abondent | Album photo classé par personne avec quelques photos annotées |
| **Auto-supervisé** | **Aucune étiquette humaine** : le label est fabriqué à partir des données | Pré-entraînement massif puis affinage (fine-tuning) supervisé | Deviner le mot masqué / le mot suivant (GPT, BERT) |

Hors programme : renforcement (récompense), transfert, few-shot / zero-shot, et semi / auto-supervisé (panorama). L'UE porte sur le **supervisé** ; K-Means (cours 2) = exemple de non supervisé.

## 2. Démarche générale

**Deux étapes, toujours** : (1) **apprentissage** (données + algorithme → modèle entraîné) : coût **élevé**, fait **une seule fois** ; (2) **prédiction** (modèle appliqué à une nouvelle donnée) : coût **faible**, fait **à chaque fois**.

**Chaîne de traitement** : données brutes → **extraction de représentation** (vecteurs) → choix et apprentissage d'une méthode → **évaluation** (généralise-t-elle ?).

| | Paramètres μ | Hyperparamètres λ |
|---|---|---|
| Fixés | Automatiquement, en minimisant un critère sur train (descente de gradient) | **Avant** l'apprentissage, par essais successifs sur **validation** |
| Exemples | Coefficients d'une droite de régression, $a_0$ … $a_D$ | Degré D, nombre de voisins k, régularisation α, **choix de la méthode** ; pour l'extraction : taille du vocabulaire, nombre de bins, composantes PCA |

**PIÈGE : POURQUOI NE PAS TOUT APPRENDRE SUR TRAIN ?**

Un critère évalué sur les **mêmes données** que celles qui servent à apprendre a toujours intérêt à devenir plus complexe : augmenter D ne peut que faire baisser l'erreur train, jusqu'à passer par tous les points. Le but est de bien marcher sur de **nouvelles** données.

## 3. Train, validation, test et validation croisée

| Ensemble | Part | Rôle |
|---|---|---|
| **Train** | 60 % | **Apprend** l'extraction de caractéristiques (dictionnaire, normalisation) **et** les paramètres du modèle |
| **Validation** | 20 % | **Réapplique sans réapprendre** ce qui a été appris sur train, mesure → choisit les **hyperparamètres** (répété) |
| **Test** | 20 % | Mis de côté dès le départ, regardé **une seule fois à la fin** avec l'extraction et le modèle finaux → performance reportée |

**PIÈGE : CHOISIR SUR LE TEST**

Garder le réglage qui a le meilleur score sur test, c'est **adapter ses choix au test** : son score n'est plus une estimation honnête, même sans avoir appris dessus.

D'où le 3e ensemble, la validation. Train et validation très différents = **signal d'alerte** (sur-apprentissage) avant même le test.

**VALIDATION CROISÉE K-FOLD**

On découpe l'ensemble d'apprentissage (sans le test) en **k parts égales**. On répète k fois : **k – 1 parts** pour entraîner, la dernière pour valider (à tour de rôle), puis on **moyenne** les k scores. Toute la donnée sert à valider ; utile avec **peu de données** ; coût : **k fois plus** d'entraînements.

**TUTO : COMPTER LES EXEMPLES ET LES ENTRAÎNEMENTS**

**Découpage 60 / 20 / 20 de N = 1200**

Train = 0,6 × 1200 = **720** · validation = **240** · test = **240**.

Seuls les 720 de train apprennent μ (pas 960 = train + validation).

**k = 4 sur 800 exemples, 5 valeurs de D à comparer**

Une part = 800 / 4 = 200. Chaque entraînement : 3 parts = **600** exemples.

Entraînements : k × nb de valeurs = 4 × 5 = **20**.

## 4. Généralisation : risque, sous / sur-apprentissage

**Risque empirique** (calculable, sur les P exemples) :

$$R_{emp}(f) = (1/P) \sum_{i=1..P} L(f(x_i), y_i)$$

**Risque réel** (sur toute la distribution, inconnue) :

$$R(f) = E_{(x,y)}[L(f(x), y)]$$

**Généraliser** : faire que $R_{emp}$ soit un bon indicateur de R. On l'estime sur des exemples qui **n'ont pas servi** à apprendre. L = taux d'erreur, EQM…

**Métrique ≠ loss** : la métrique la plus parlante n'est pas toujours dérivable, donc on entraîne sur une autre loss et on valide sur la métrique.

**Exemple** : $\hat{y}$ = (1,5 ; 2 ; 2 ; 4), y = (1 ; 2 ; 3 ; 4), L = $(\hat{y} - y)^2$

Carrés : 0,25 ; 0 ; 1 ; 0 → somme 1,25 → $R_{emp}$ = 1,25 / 4 = **0,3125**

| | Sous-apprentissage | Sur-apprentissage |
|---|---|---|
| Cause | Modèle trop simple (ex. droite sur données non linéaires) | Modèle trop complexe, peu d'exemples : apprend **par cœur** le bruit |
| Erreur train | **Élevée** | Très faible |
| Erreur validation | Élevée | **Élevée** (grand écart avec train) |
| Courbe validation | À gauche du creux | À droite du creux |
| Plus de données ? | **Ne corrige pas** (D = 1 reste à 0,65) | **Compense** la complexité (D = 16 : 2 500 → 0,03) |

**CHOISIR LE DEGRÉ**

Le bon degré est au **minimum de l'erreur de validation**, jamais sur l'erreur train (baisse toujours) ni sur le test (réservé à la fin).

**MALÉDICTION DE LA DIMENSION**

En haute dimension, les données deviennent **éparses** : le volume à couvrir croît **exponentiellement** avec N. Avec 10 points par axe, il faut $\mathbf{10^N}$ points ($10^6$ en dimension 6, $10^{10}$ en dimension 10). Les méthodes de **voisinage** (k plus proches voisins) en souffrent le plus. Plus de features ⇒ plus de données nécessaires.

## 5. Représenter les données (vectorisation, embedding)

La plupart des algorithmes veulent un vecteur $\mathbf{x} \in \mathbb{R}^N$.
Une bonne représentation :

- a une **taille fixe N**, quelle que soit la donnée brute ;
- **préserve l'information utile** pour la tâche ;
- a une **géométrie qui a du sens** : données similaires → vecteurs proches, pas de fausse relation (ordre, distance, magnitude).

| Donnée | Représentations citées |
|---|---|
| Catégorie | **One-hot** (base canonique) |
| Texte | Nombre de mots, longueur moyenne (perdent beaucoup) ; **bag-of-words** (vocabulaire fixé à l'avance) |
| Image | Histogramme des **couleurs** (taille indépendante de la résolution) ; histogramme du **gradient** (contours, formes) |
| Son | **Spectre des fréquences** sur une fenêtre de temps fixe |

**ONE-HOT PLUTÔT QUE 1, 2, 3…**

```
XS = (1,0,0,0,0)    S  = (0,1,0,0,0)
M  = (0,0,1,0,0)    L  = (0,0,0,1,0)
XL = (0,0,0,0,1)
```

Coder XS = 1 … XL = 5 imposerait des **écarts et une magnitude** (« XL = 5 × XS ») ; faux pour une catégorie sans ordre (couleur).

**Distance entre deux catégories** : deux one-hot différents diffèrent sur 2 coordonnées → $\sqrt{(1^2 + 1^2)} = \sqrt{2} \approx 1{,}41$ pour **toute** paire : catégories équidistantes.

## 6. Formulation et optimisation

$\hat{y} = f_{\mu,\lambda}(x)$ : $x \in \mathbb{R}^N$ donnée, $\hat{y} \in \mathbb{R}$ ou $\{-1, 1\}$, **μ paramètres**, **λ hyperparamètres**.

$g(f_{\mu,\lambda}, \{X, Y\})$ : mesure à quel point f se trompe sur les données d'apprentissage (version calculable de $R_{emp}$).

$\mu^* = \text{argmin}_\mu\, g(f_{\mu,\lambda}, \{X, Y\})$ : apprendre = minimiser g par rapport à μ (λ fixé).

| Convexe | Non convexe |
|---|---|
| Simple à analyser et résoudre, algorithme générique efficace, **unique minimum global** | Difficile, pas d'algorithme générique, **plusieurs minima** locaux / globaux |

En pratique : pas de formule fermée, beaucoup de paramètres → méthode **itérative** (descente de gradient, TD 1).

**RASOIR D'OCKHAM ET RÉGULARISATION**

Même en convexe, trop de paramètres / trop peu de données ⇒ infinité de solutions. Entre solutions équivalentes, **préférer la plus simple** (moins de variables, petits coefficients). Deux leviers : un modèle assez simple (sans sous-apprendre), ou un modèle riche **régularisé** : ajouter $\alpha\|\mu\|_2^2$ au critère.

**EXEMPLE FINAL : RÉGRESSION POLYNOMIALE**

```
f(x) = a0 + a1·x + … + aD·x^D
μ = {a0, …, aD}    → D + 1 paramètres (train)
λ = {D, α}         → choisis sur validation
μ* = argmin  ∑ (f(xi) − yi)²  +  α‖μ‖²
```

D = 5 → **6 paramètres** (ne pas oublier $a_0$). α trop **faible** → sur-apprentissage (comme D trop grand) ; α trop **fort** → sous-apprentissage (comme D trop petit). Le test ne fait que **confirmer a posteriori**.

## 7. Évaluer un classifieur

Le **taux d'erreur** (fausses prédictions / prédictions) est trop générique et **trompeur** : avec 990 positifs et 10 négatifs, répondre toujours « positif » donne **1 %** d'erreur mais une spécificité de **0 %**.

| | Prédit Faux | Prédit Vrai |
|---|---|---|
| Vérité Faux | TN | FP |
| Vérité Vrai | FN | TP |

| Critère | Formule | Quand l'utiliser |
|---|---|---|
| Exactitude (accuracy) | (TP + TN) / total | Trompeuse si classes déséquilibrées |
| **Précision** | TP / (TP + FP) | Un **faux positif** coûte cher (anti-spam) |
| **Rappel** = sensibilité | TP / (TP + FN) | **Rater un positif** coûte cher (diagnostic) |
| Spécificité | TN / (TN + FP) | Taux de vrais négatifs |
| $F_1$ | 2PR / (P + R) | Moyenne **harmonique** de P et R |
| $F_\beta$ | — | **β > 1 → rappel**, β < 1 → précision |

Pas adaptés tels quels au **multi-label** : calcul par label, puis moyenne.

**TUTO : LES DEUX TESTS DE TUMEUR (POSITIF = MALIGNE)**

| Test 1 | préd. mal. | préd. bén. |
|---|---|---|
| vraie mal. | 98 | 2 |
| vraie bén. | 50 | 850 |

| Test 2 | préd. mal. | préd. bén. |
|---|---|---|
| vraie mal. | 55 | 45 |
| vraie bén. | 5 | 895 |

| | Test 1 | Test 2 |
|---|---|---|
| Exactitude | 948/1000 = 94,8 % | 950/1000 = **95,0 %** |
| Précision | 98/148 = 66,2 % | 55/60 = **91,7 %** |
| Rappel | 98/100 = **98 %** | 55/100 = 55 % |
| Spécificité | 850/900 = 94,4 % | 895/900 = 99,4 % |
| $F_1$ | **79,0 %** | 68,8 % |

Le test 2 gagne en exactitude mais **rate 45 tumeurs sur 100** : pour un diagnostic, on préfère le **test 1** (rappel).

**PIÈGES DE CALCUL**

- Précision : on divise par la **colonne** « prédit positif » ; rappel : par la **ligne** « vrai positif ».
- MNIST (lignes = vraie classe) : rappel du 8 = 853 / 975 = **87,5 %**, précision = 853 / 956 = **89,2 %** ; exactitude = 9 272 / 10 000 = 92,7 %. Confusion max : **vrai 4 prédit 9** (36).
- $F_1$ = moyenne **harmonique** : test 2 → 68,8 %, pas 73,3 %.

**COURBES ET AUTRES CRITÈRES**

- **Rappel-précision** : on fait varier le **seuil** du score ; précision et rappel évoluent en sens opposé. Très informative si classes **déséquilibrées** ; résumée par son aire = **précision moyenne** (binaire et multi-label, pas multi-classes).
- **ROC** : sensibilité en fonction de **1 – spécificité**, pour tous les seuils → compare **indépendamment du seuil**. **AUC** : 1 = parfait, **0,5 = aléatoire**. Peut être trop optimiste sur classes très déséquilibrées.
- **MCC** (Matthews) : toute la matrice en un chiffre entre **–1 et 1**, robuste au déséquilibre.

**NO FREE LUNCH (WOLPERT & MACREADY)**

Sur **tous les problèmes possibles**, aucun algorithme n'est meilleur qu'un autre (ni que l'aléatoire). Les vrais problèmes sont structurés : connaître **plusieurs méthodes**.

## 8. Annexe : biais et variance

**$EQM(\hat{y}) = E[(\hat{y} - y)^2] = E[(E(\hat{y}) - y)^2] + E[(E(\hat{y}) - \hat{y})^2]$ = Biais² + Var**

**Biais** = décalage du barycentre des prédictions par rapport à la vraie valeur → fort = modèle **trop simple** (sous-apprentissage). **Variance** = dispersion des prédictions d'un jeu d'entraînement à l'autre → forte = modèle **trop complexe**. Objectif : **faible biais et faible variance**.

**EXEMPLE**

Biais = 0,3 et Var = 0,05 → EQM = 0,3² + 0,05 = 0,09 + 0,05 = **0,14**.
Piège : 0,35 = biais oublié au carré.